付 録A　三輪車とトレーラーのシミュレータについて

このシミュレータは，“SICE セミナー —現代制御理論入門—” の演習用教材として，新たに開発されたもので，ノンホロノミックシステムである三輪車とトレーラーの数値シミュレーションを行い，そのシミュレーション結果を可視化するためのプログラムである．

A.1　動作環境

本プログラムでは，以下に示すプログラムを利用している．

1. Java Runtime Environment
   本プログラムは JDK(Java 2 Development Kit) 1.2.2 の環境で開発されたものである．実行するためには，JRE(Java 2 Runtime Environment) 1.2.2 以上の環境を必要とする．JRE 及び JDK に関する情報およびソフトウェアのダウンロードは，Sun Microsystems (株) の WEB ページ

   `http://jp.sun.com/java`

   から参照することができる．

2. MATX
   MATX は科学や工学に必要な数値および数式計算をサポートする記述性に優れたプログラミング言語である．東京工業大学にて開発され，主にシステム理論 (制御系の解析・設計・シミュレーション) や信号処理の研究に使われてきた．文法のうち制御の流れや関数などのプログラム構造は C 言語とほぼ同じである．行列の演算や成分の操作法は市販の計算ソフトウェア MATLAB とほぼ同じである．このプログラムに関する情報およびダウンロードは，MATX の WEB ページ

   `http://www.matx.org`

   から参照することができる．
   (MATX 本体は，著作権を放棄していないフリーソフトウェアである．)

配布する CD-ROM 内には，Windows, Solaris(Sparc, Intel), Linux(Intel) 用の，シミュレータプログラム，JRE および MATX 実行ファイルをまとめたアーカイブ (LHA もしくは TAR+GZIP) を用意しているので，このアーカイブファイルをハードディスク上で展開することで，シミュレーションを実行することができる．

本シミュレータは Java 言語に基づいて開発されているため，様々なオペレーティングシステムで実行することがでる．しかし，数値シミュレーションに MATX という外部プログラムを使用しているため，外部プログラムは適当な OS に調整する必要がある．

A.2　概要

本シミュレータは 4 つのパネルから構成され，それぞれ，アニメーションパネル，ログパネル，2D プロットパネル，および設定パネルと呼ばれる．シミュレータの全画面を Fig.A.1 に示す．ウィンドウの一番上には，プログラム，設定，描画，グラフ，ヘルプの 5 つのプルダウンメニューが用意されており，このメニューを選ぶことによって様々な形でシミュレーション結果を見ることが出来る．

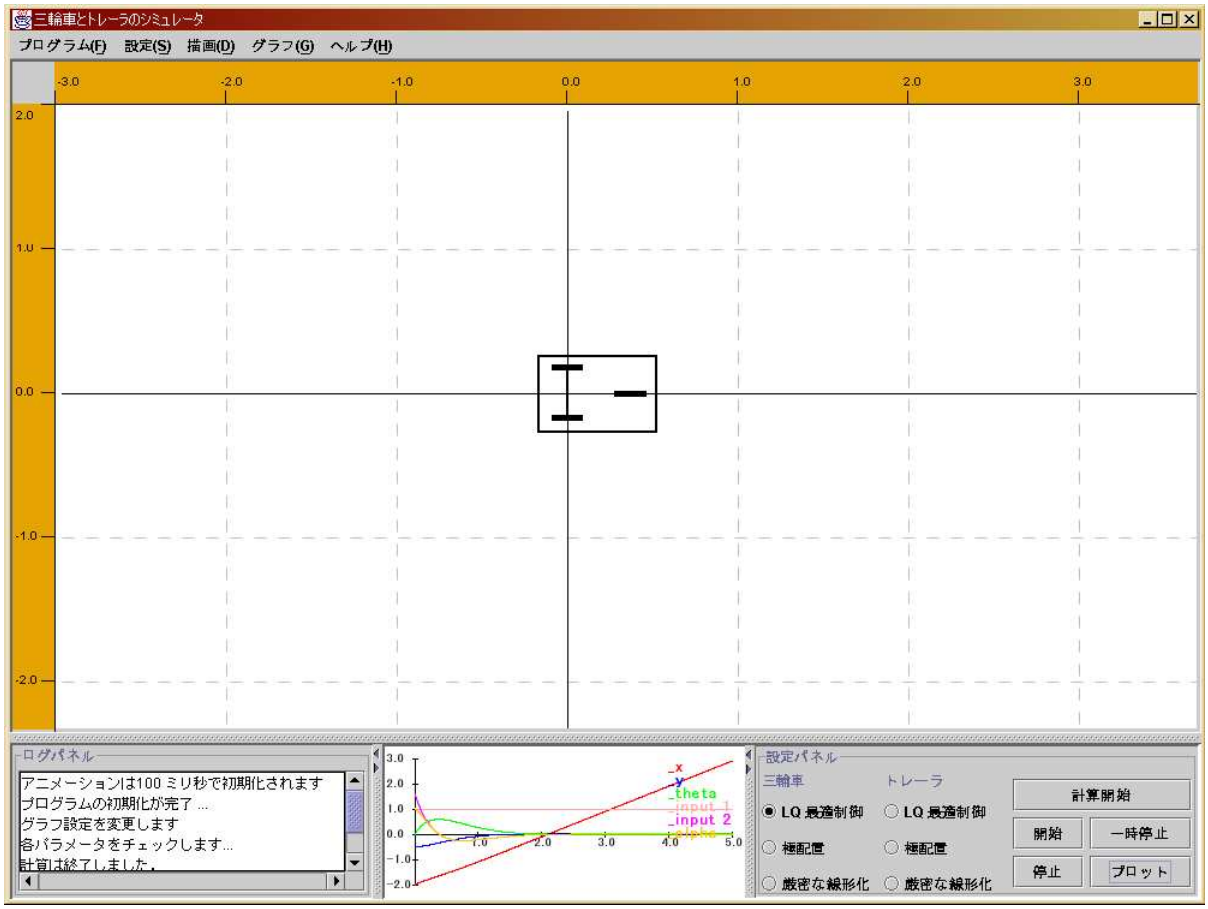

Fig. A.1: シミュレータの全画面

Fig.A.2, Fig.A.3 に示すように，アニメーションパネルでは三輪車またはトレーラーのシミュレーション結果を，アニメーションまたはスナップショットの形で描くことができる．

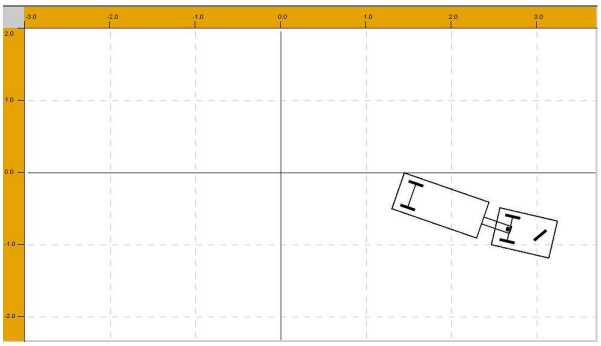

Fig. A.2: アニメーション用の画面

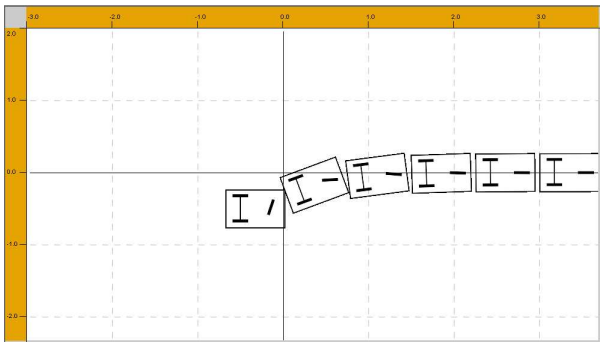

Fig. A.3: スナップショット用の画面

また，Fig.A.4 に示すように，ログパネルでは現在実行されている状況が表示される．エラーやシミュレータの実行記録に関する情報もこのパネルに表示される．さまざまなメニューを実行する度に，必ずこのパネルにメッセージが出力される．

Fig.A.5 に示す 2D プロットパネルでは，シミュレーション結果の時系列グラフを表示することができる．表示するデータの種類はグラフメニューから選択することができる．

最後に，Fig.A.6 に示すように，設定パネルでは三輪車やトレーラーに関するシミュレーションの種類を選択するボタンが置かれている．どのようなシミュレーションを行うかを必ず先に選択しなければならない．例えば，三輪車の LQ 最適制御のシミュレーションを行う場合には，まず三輪車での LQ 最適制御のボタンを選択する．また，数値シミュレーションを開始する計算開始ボタンや，開始ボタン，一時停止ボタン，停止ボタンのようなシミュレーション結果の可視化を制御するボタン，2D プロットを実行するボタンもこのパネルに置かれている．

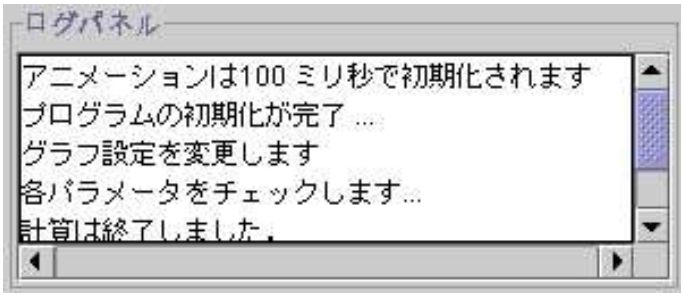

Fig. A.4: ログ表示の画面

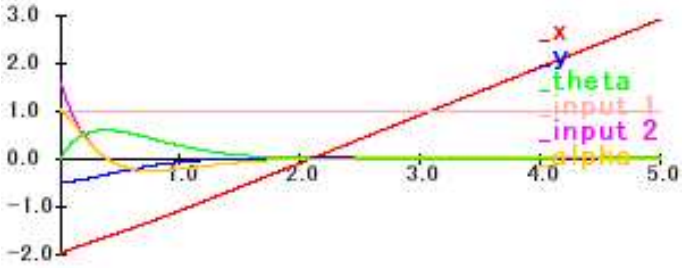

Fig. A.5: 2D プロットの画面

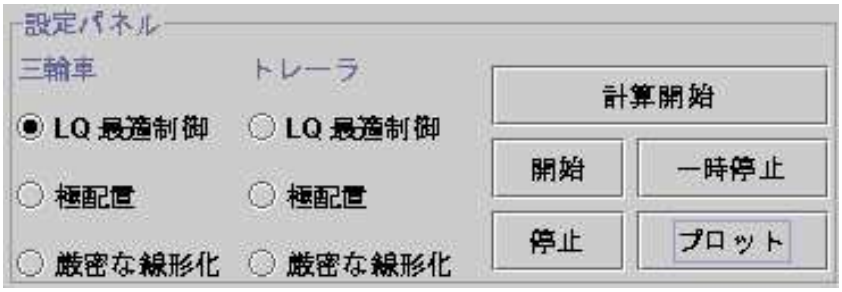

Fig. A.6: 各シミュレーション用の設定画面

A.3 プルダウンメニューの機能

A.3.1 プログラムメニュー

プログラムメニューでは Fig.A.7 に示すように，次のような機能を持っている．

計算開始 これを選択すると，外部プログラム (MATX プログラム) が実行され，数値シミュレーションの計算が始まる．これは設定パネルにある計算開始ボタンと同じ機能である．

開始 これを選択すると，シミュレーションの可視化が始まる．これは設定パネルにある開始ボタンと同じ機能である．

一時停止 これを選択すると，シミュレーションの可視化が一時停止される．再び選択すると，シミュレーション可視化の続きが再び始まる．これは設定パネルにある一時停止ボタンと同じ機能である．

アニメーション停止 これを選択すると，シミュレーションの可視化が停止される．これは設定パネルにある停止ボタンと同じ機能である．

プロット これを選択すると，グラフメニューで選択された時系列データに関し 2D プロットを行う．これは設定パネルにあるプロットボタンと同じ機能である．

プログラム終了 これを選択すると，シミュレータプログラムが終了する．

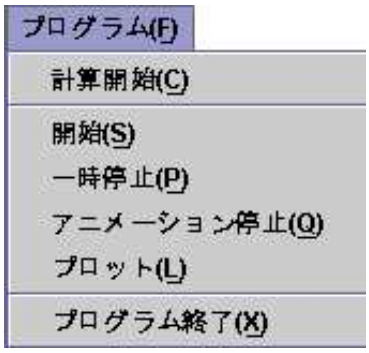

Fig. A.7: プログラムメニュー

A.3.2 設定メニュー

設定メニューでは Fig.A.8 に示すように，次のような機能を持っている．

縮小 これを選択すると，アニメーションパネルの座標間隔が小さくなり，一回選択する毎にアニメーション画面が半分に縮小される．最小の画面の制限はない．

拡大 これを選択すると，アニメーションパネルの座標間隔が大きくなり，一回選択する毎にアニメーション画面が 2 倍に拡大される．最大のサイズには制限があり，最大の画面はプログラムが起動された時の画面のサイズである．

パラメータ変更 これを選択すると，Fig.A.9 に示すような新しいウィンドウが開き，数値シミュレーションを行う際の各パラメータを変更することができる．変更できるパラメータとしては，初期値，シミュレーション時間，および LQ 最適制御の時には行列 Q と R のパラメータ，極配置の時には極のパラメータを変更できる．

アニメーション速度変更 これを選択すると，Fig.A.10 に示すような新しいウィンドウが開き，アニメーションの速度やスナップショットの間隔を調整することができる．アニメーションの時に値を大きくすると，アニメーションが早く動くことになる．スナップショットの時に値を大きくすると，スナップショットの時間間隔が大きくなる．

Fig. A.8: 設定メニュー

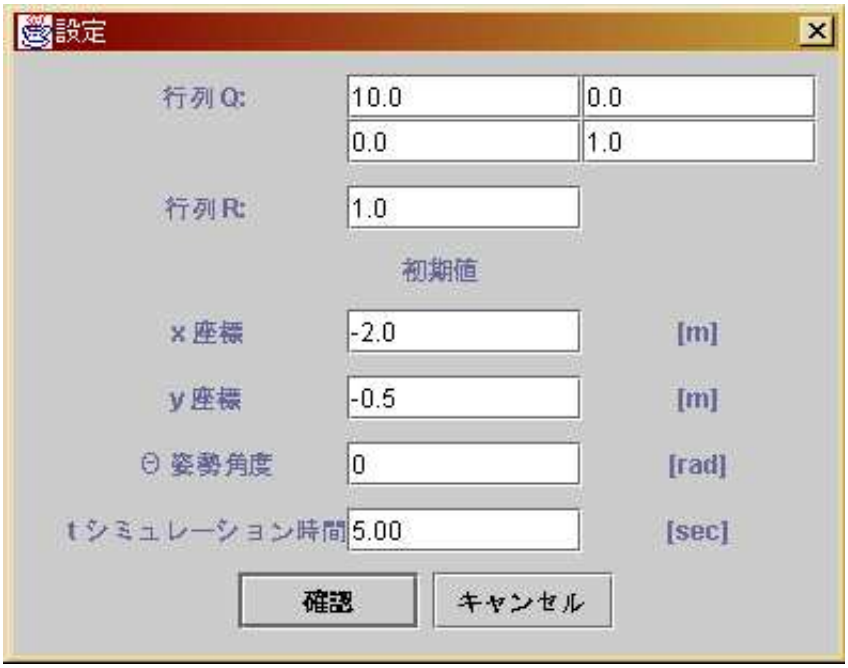

Fig. A.9: パラメータ変更ウィンドウ

Fig. A.10: アニメーション速度変更ウィンドウ

A.3.3　描画メニュー

描画メニューでは Fig.A.11 に示すように，次のような機能を持っている．

アニメーションの作成　これを選択すると，シミュレーション結果を動画の形で見ることができる．

スナップショットの作成　これを選択すると，シミュレーション結果の軌跡がアニメーション速度変更ウィンドウで選択された間隔で描かれる．

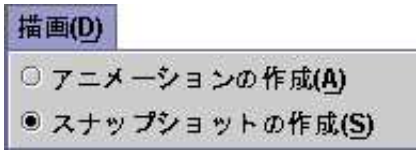

Fig. A.11: 描画メニュー

A.3.4　グラフメニュー

グラフメニューでは Fig.A.12 に示すように，次のような機能を持っている．

グラフ選択　これを選択すると，Fig.A.13 に示すような新しいウィンドウが開き，2D プロットで描画したい時系列データを選択することができる．三輪車のシミュレーション実行時とトレーラーのシミュレーション実行時では選択できる変数が異なる．データは複数選択可能である．

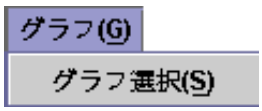

Fig. A.12: グラフメニュー

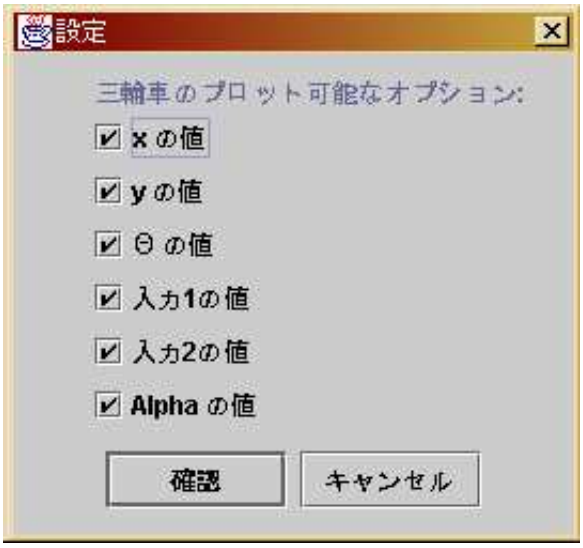

Fig. A.13: プロットデータ選択ウィンドウ

A.3.5 ヘルプメニュー

ヘルプメニューでは Fig.A.14 に示すように，次のような機能を持っている．

このシミュレータについて これを選択すると新しいウィンドウが開き，本シミュレータに関する情報が表示される．

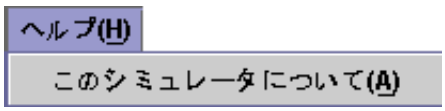

Fig. A.14: ヘルプメニュー

A.4 ディレクトリ構成

本シミュレータは次のようなディレクトリ構成 (Windows の場合) となっている．

- `Windows`
  - `Simulator`
    - `Simulator.bat` ··· シミュレーションプログラム用のスクリプト (バッチファイル)
    - `Simulator.jar` ··· シミュレーションのクラスライブラリ
    - `flag.mm` ··· シミュレーションフラグファイル
    - `trail.exe` ··· トレーラー用のシミュレーションプログラム (MATX 実行ファイル)
    - `trailer.gif` ··· トレーラー用の画像ファイル
    - `tri.exe` ··· 三輪車用のシミュレーションプログラム (MATX 実行ファイル)
    - `tricycle.gif` ··· 三輪車用の画像ファイル
    - `tyre.gif` ··· 三輪車のタイヤの画像
    - `src` ··· 数値シミュレーションプログラム (MATX) のソースファイルディレクトリ
    - `trailer` ··· トレーラーのシミュレーション結果とパラメータを保存するためのディレクトリ
      - `Q.mat` ··· 行列 Q のデータファイル
      - `R.mat` ··· 行列 R のデータファイル
      - `data.mat` ··· シミュレーション結果のデータファイル
      - `init.mat` ··· 初期値のデータファイル
      - `pole.mat` ··· 極のデータファイル
      - `tf.mat` ··· シミュレーション時間のデータファイル
    - `tricycle` ··· 三輪車のシミュレーション結果とパラメータを保存するためのディレクトリ
      - `Q.mat` ··· 行列 Q のデータファイル
      - `R.mat` ··· 行列 R のデータファイル
      - `data.mat` ··· シミュレーション結果のデータファイル
      - `init.mat` ··· 初期値のデータファイル
      - `pole.mat` ··· 極のデータファイル
      - `tf.mat` ··· シミュレーション時間のデータファイル
  - `j2re1.4.1_02` ··· Java プログラムを実行するためのプログラムディレクトリ (JRE)

注:Solaris や Linux の場合は，シミュレーションプログラム用のスクリプトファイル名は `Simulator.sh` となっている．

A.5　三輪車のシミュレーション

Fig.A.15 のような三輪車のモデルを考える.

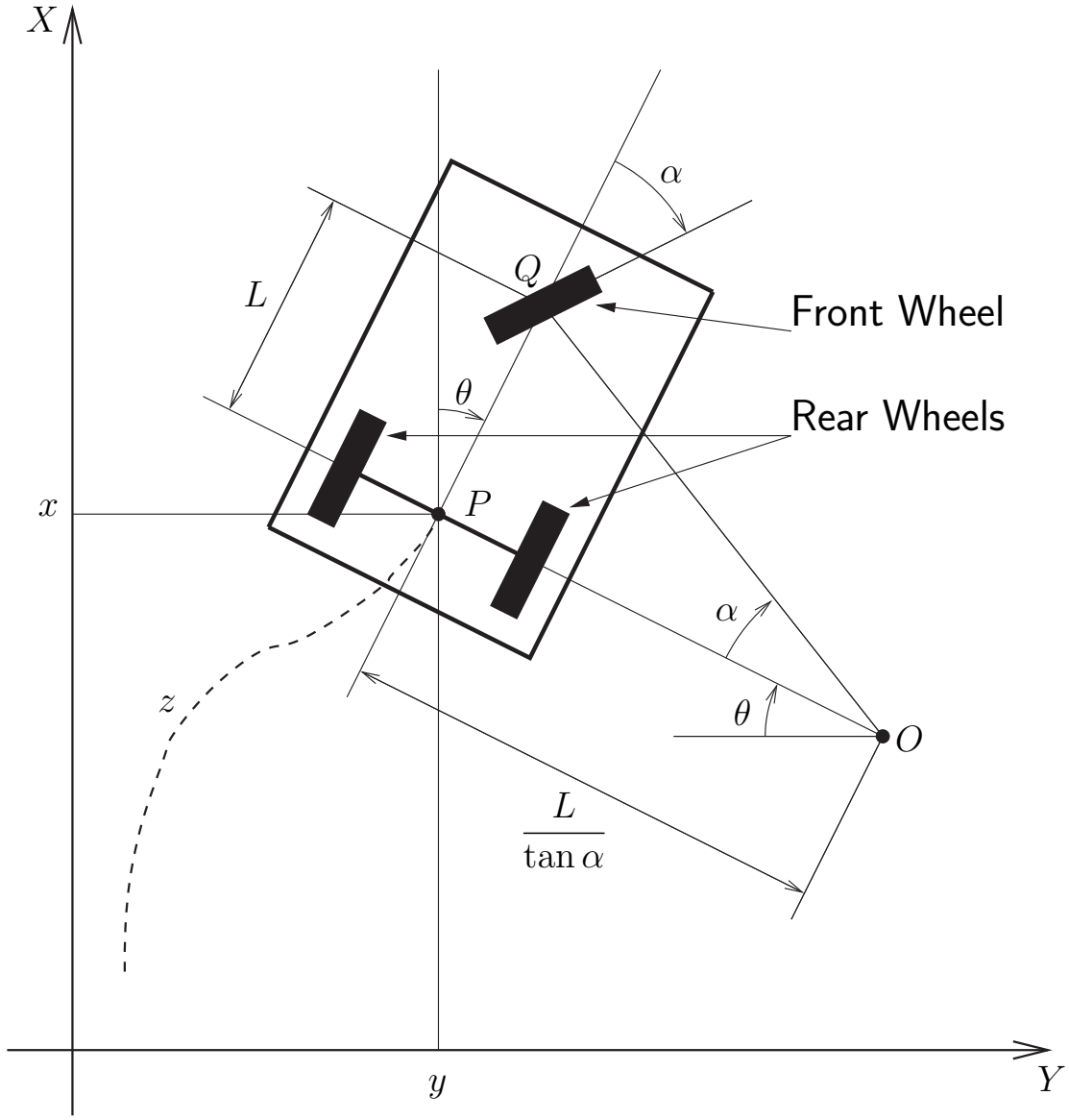

Fig. A.15: 三輪車モデル

Table A.1: 記号の定義

P	:	2 つの後輪の中点
O	:	前輪の回転軸と後輪の回転軸の交点 (回転中心)
Q	:	操舵軸 ($OP \perp PQ$)
L	:	PQ 間の距離 (0.4 [m])
θ	:	車両の姿勢角 [rad]
α	:	操舵角 [rad]
z	:	三輪車の軌跡

Fig.A.15 を基に，三輪車の運動方程式は次のように表すことができる．

$$\frac{d}{dt}\begin{pmatrix} x \\ y \\ \theta \end{pmatrix} = \begin{pmatrix} \cos\theta \\ \sin\theta \\ 0 \end{pmatrix}\dot{z} + \begin{pmatrix} 0 \\ 0 \\ \frac{1}{L} \end{pmatrix}\dot{z}\tan\alpha \tag{A.1}$$

この運動方程式の新しい時間軸を x とすると，次のように変形することができる．

$$\begin{cases} \dfrac{d}{dx}\begin{pmatrix} y \\ \theta \end{pmatrix} = \begin{pmatrix} \tan\theta \\ 0 \end{pmatrix} + \begin{pmatrix} 0 \\ \frac{1}{L\cos\theta} \end{pmatrix}\tan\alpha \\ \dfrac{dx}{dt} = \cos\theta\dot{z} \end{cases} \tag{A.2}$$

x が単調増加するとき，車両は前進し X 軸に追従する．よって，この x を時間軸と考えて制御を行う．これは (A.2) の第 2 式を用いて $\frac{dx}{dt}$ と制御していると考えてもよい．制御方策は，$(y,\theta)^T = ($目標起動との偏差，姿勢$)^T$ と捉えて $y,\theta \to 0$ に追従させるようなことを実現すればよい．

このシステムのシミュレーション方法を以下に述べる．但し，シミュレーションでは PQ 間の距離を $L = 0.4$[m] としている．

case 1 システム方程式を原点近傍で線形化を行い，LQ 制御と極配置による線形制御理論に基づく状態フィードバックを作る．

新しい時間軸 x に関する方程式を原点近傍で線形化を行うと，

$$\begin{aligned} f(x) &\stackrel{\text{def}}{=} \begin{pmatrix} \tan\theta \\ 0 \end{pmatrix} \Rightarrow A \stackrel{\text{def}}{=} \begin{pmatrix} \frac{\partial f_1}{\partial y} & \frac{\partial f_1}{\partial \theta} \\ \frac{\partial f_2}{\partial y} & \frac{\partial f_2}{\partial \theta} \end{pmatrix} \\ &= \begin{pmatrix} 0 & \frac{1}{\cos^2\theta} \\ 0 & 0 \end{pmatrix}\Bigg|_{(y,\theta)=(0,0)} = \begin{pmatrix} 0 & 1.0 \\ 0 & 0 \end{pmatrix} \\ g(x) &\stackrel{\text{def}}{=} \begin{pmatrix} 0 \\ \frac{1}{L\cos\theta} \end{pmatrix} \Rightarrow B \stackrel{\text{def}}{=} \begin{pmatrix} 0 \\ \frac{1}{L} \end{pmatrix} \end{aligned}$$

より，線形方程式は次のようになる．ここでは $\tan\alpha$ を入力と考えている．

$$\frac{d}{dx}\begin{pmatrix} y \\ \theta \end{pmatrix} = \begin{pmatrix} 0 & 1.0 \\ 0 & 0 \end{pmatrix}\begin{pmatrix} y \\ \theta \end{pmatrix} + \begin{pmatrix} 0 \\ \frac{1}{0.4} \end{pmatrix}\tan\alpha \tag{A.3}$$

LQ 制御と極配置によって，この線形システムの状態フィードバックを求めることができる．

case 2 システム方程式を厳密な線形化を用いて線形システム方程式を作り，得られた線形方程式に対して極配置によって状態フィードバックを求める．

新しい状態 ϕ と新しい入力 v として，

$$\begin{cases} \phi = \begin{pmatrix} \phi_1 \\ \phi_2 \end{pmatrix} = \begin{pmatrix} y \\ \tan\theta \end{pmatrix} \\ \tan\alpha = L\cos^3\theta v \end{cases}$$

を用いると次の線形方程式が得られる．

$$\frac{d\phi}{dt} = \begin{pmatrix} 0 & 1.0 \\ 0 & 0 \end{pmatrix}\phi + \begin{pmatrix} 0 \\ 1.0 \end{pmatrix}v \tag{A.4}$$

極配置によって，この線形システムの状態フィードバックを求めることができる．

A.6 トレーラーのシミュレーション

Fig.A.16 のようなトレーラーのモデルを考える.

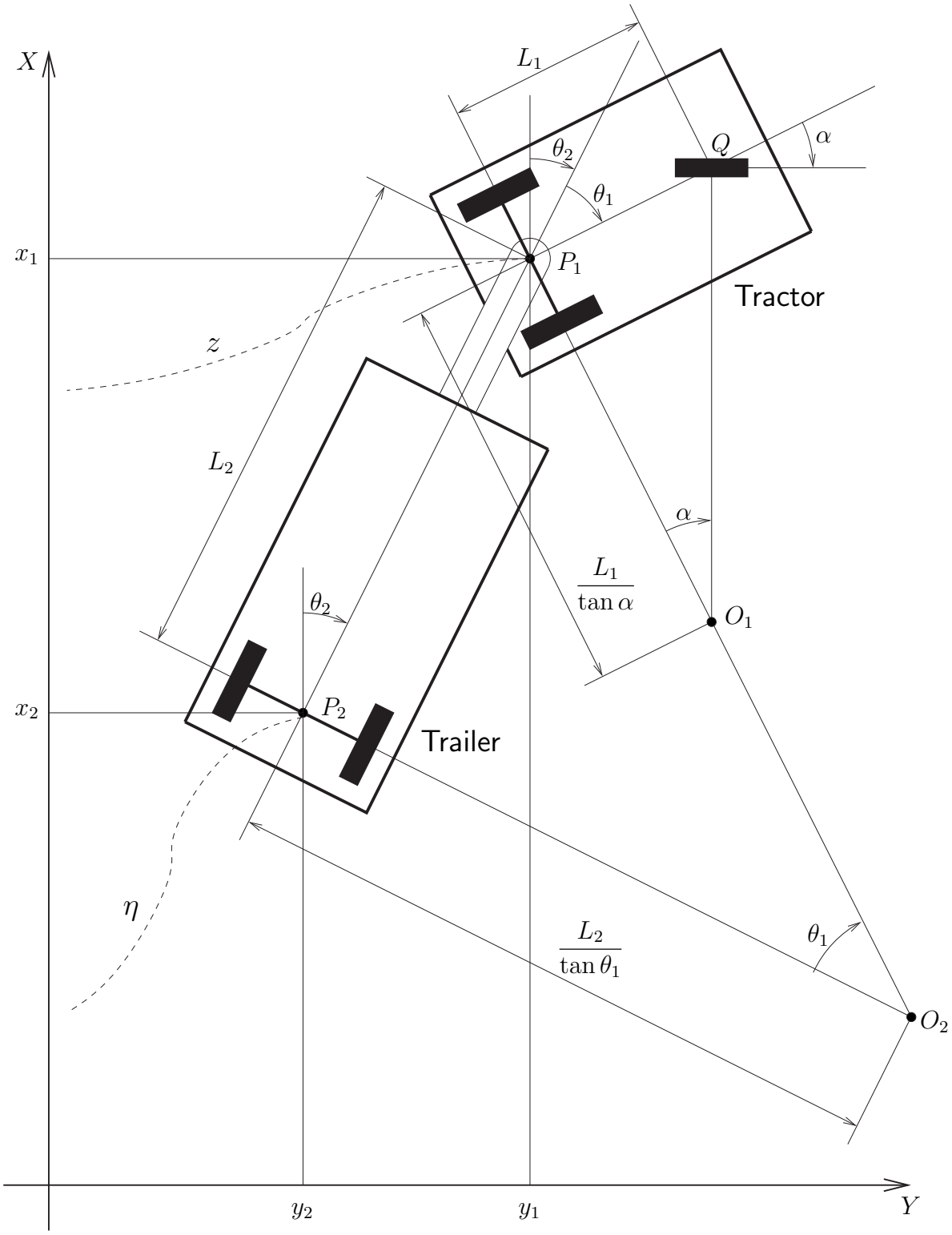

Fig. A.16: トレーラーモデル

Table A.2: トラクターに関する記号の定義

P_1	:	2 つの後輪の中点
O_1	:	前輪の回転軸と後輪の回転軸の交点 (トラクターの回転中心)
Q	:	操舵軸 ($O_1P_1 \perp P_1Q$)
L_1	:	P_1Q 間の距離 (0.4 [m])
θ_1	:	トレーラに対するトラクターの姿勢角
α	:	操舵角
z	:	トラクターの軌跡

Table A.3: トレーラーに関する記号の定義

P_2	:	2 つの後輪の中点
O_2	:	P_1 と後輪の回転軸の交点 (トレーラーの回転中心)
Q	:	操舵軸 ($O_1P_1 \perp P_1Q$)
L_2	:	P_1P_2 間の距離 (1.2 [m])
θ_2	:	トレーラーの姿勢角
η	:	トレーラーの軌跡

Fig.A.16 を基に，トレーラーの運動方程式は次のように表すことができる．

$$\frac{d(\theta_1+\theta_2)}{dt} = \dot{z}\frac{\tan\alpha}{L_1} \tag{A.5}$$

$$\frac{d\theta_2}{dt} = \dot{\eta}\frac{\tan\theta_1}{L_2} \tag{A.6}$$

$$\frac{dx_2}{dt} = \dot{\eta}\cos\theta_2 \tag{A.7}$$

$$\frac{dy_2}{dt} = \dot{\eta}\sin\theta_2 \tag{A.8}$$

$$\frac{d\eta}{dt} = \dot{z}\cos\theta_1 \tag{A.9}$$

この運動方程式の新しい時間軸 (トレーラーの目標軌道上の位置) を x_2 とすると，次のようになる．

$$\left\{\begin{array}{rcl} \dfrac{d}{dx_2}\begin{pmatrix} y_2 \\ \theta_1 \\ \theta_2 \end{pmatrix} & = & \begin{pmatrix} \tan\theta_2 \\ -\frac{\tan\theta_1}{L_2\cos\theta_2} \\ \frac{\tan\theta_1}{L_2\cos\theta_2} \end{pmatrix} + \begin{pmatrix} 0 \\ \frac{1}{L_1\cos\theta_1\cos\theta_2} \\ 0 \end{pmatrix}\tan\alpha \\ \dfrac{dx_2}{dt} & = & \dot{\eta}\cos\theta_2 \end{array}\right. \tag{A.10}$$

制御方策は，このシステムにおいて $(y_2, \theta_1, \theta_2)^T =$ (目標軌道との偏差，トラクター姿勢角，トレーラー姿勢角)^T と捉えて，$y_2, \theta_1, \theta_2 \to 0$ に追従させることを実現すればよい．

このシステムのシミュレーション方法を以下に述べる．但し，シミュレーションではトラクターとトレーラー軸の長さはそれぞれ $L_1 = 0.4$[m] と $L_2 = 1.2$[m] としている．

case 1 システム方程式を原点近傍で線形化を行い，LQ 制御と極配置による線形制御理論に基づく状態フィードバックを作る．

新しい時間軸 x に関する方程式を原点近傍で線形化を行うと，

$$
\begin{aligned}
f(x) &\stackrel{\text{def}}{=} \begin{pmatrix} \tan\theta_2 \\ -\frac{\tan\theta_1}{L_2\cos\theta_2} \\ \frac{\tan\theta_1}{L_2\cos\theta_2} \end{pmatrix} \quad \Rightarrow \quad A \stackrel{\text{def}}{=} \begin{pmatrix} \frac{\partial f_1}{\partial y_2} & \frac{\partial f_1}{\partial \theta_1} & \frac{\partial f_1}{\partial \theta_2} \\ \frac{\partial f_2}{\partial y_2} & \frac{\partial f_2}{\partial \theta_1} & \frac{\partial f_2}{\partial \theta_2} \\ \frac{\partial f_3}{\partial y_2} & \frac{\partial f_3}{\partial \theta_1} & \frac{\partial f_3}{\partial \theta_2} \end{pmatrix} \\
&= \left. \begin{pmatrix} 0 & 0 & \frac{1}{\cos^2\theta_2} \\ 0 & \frac{-1}{L_2\cos^2\theta_1\cos\theta_2} & \frac{\tan\theta_1}{L_2\cos^2\theta_2} \\ 0 & \frac{1}{L_2\cos^2\theta_1\cos\theta_2} & \frac{-\tan\theta_1}{L_2\cos^2\theta_2} \end{pmatrix} \right|_{(y_2,\theta_1,\theta_2)=(0,0,0)} \\
&= \begin{pmatrix} 0 & 0 & 1.0 \\ 0 & \frac{-1}{L_2} & 0 \\ 0 & \frac{1}{L_2} & 0 \end{pmatrix} \\
g(x) &\stackrel{\text{def}}{=} \begin{pmatrix} 0 \\ \frac{1}{L_1\cos\theta_1\cos\theta_2} \\ 0 \end{pmatrix} \quad \Rightarrow \quad B \stackrel{\text{def}}{=} \begin{pmatrix} 0 \\ \frac{1}{L_1} \\ 0 \end{pmatrix}
\end{aligned}
$$

より，線形方程式は次のようになる．ここでは $\tan\alpha$ を入力と考えている．

$$
\frac{d}{dx_2}\begin{pmatrix} y_2 \\ \theta_1 \\ \theta_2 \end{pmatrix} = \begin{pmatrix} 0 & 0 & 1.0 \\ 0 & -\frac{1}{1.2} & 0 \\ 0 & \frac{1}{1.2} & 0 \end{pmatrix}\begin{pmatrix} y_2 \\ \theta_1 \\ \theta_2 \end{pmatrix} + \begin{pmatrix} 0 \\ \frac{1}{0.4} \\ 0 \end{pmatrix}\tan\alpha \tag{A.11}
$$

このシミュレーションではトレーラが後退するとしているので，$x_2' = -x_2$ を用いて，線形方程式は次のようになる．

$$
\frac{d}{dx_2'}\begin{pmatrix} y_2 \\ \theta_1 \\ \theta_2 \end{pmatrix} = \begin{pmatrix} 0 & 0 & -1.0 \\ 0 & \frac{1}{1.2} & 0 \\ 0 & -\frac{1}{1.2} & 0 \end{pmatrix}\begin{pmatrix} y_2 \\ \theta_1 \\ \theta_2 \end{pmatrix} + \begin{pmatrix} 0 \\ -\frac{1}{0.4} \\ 0 \end{pmatrix}\tan\alpha \tag{A.12}
$$

LQ 制御と極配置によって，この線形システムの状態フィードバックを求めることができる．

case 2 システム方程式を厳密な線形化を用いて線形システム方程式を作り，得られた線形方程式に対して極配置によって状態フィードバックを求める．
新しい状態 ϕ と新しい入力 v として，

$$
\begin{cases}
\phi = \begin{pmatrix} \phi_1 \\ \phi_2 \\ \phi_3 \end{pmatrix} = \begin{pmatrix} y_2 \\ \tan\theta_2 \\ \frac{\tan\theta_1}{L_2\cos^3\theta_2} \end{pmatrix} \\
\tan\alpha = -\dfrac{L_1\cos\theta_1(3\sin^2\theta_1\tan\theta_2 - \tan\theta_1)}{L_2} + L_1L_2\cos^3\theta_1\cos^4\theta_2 v
\end{cases}
$$

を用いると次の線形方程式が得られる．

$$
\frac{d\phi}{dt} = \begin{pmatrix} 0 & 1.0 & 0 \\ 0 & 0 & 1.0 \\ 0 & 0 & 0 \end{pmatrix}\phi + \begin{pmatrix} 0 \\ 0 \\ 1.0 \end{pmatrix} v \tag{A.13}
$$

このシミュレーションではトレーラが後退するとしているので，$x_2' = -x_2$ を用いて，線形方程式は次のようになる．

$$
\frac{d\phi}{dt} = \begin{pmatrix} 0 & -1.0 & 0 \\ 0 & 0 & -1.0 \\ 0 & 0 & 0 \end{pmatrix}\phi + \begin{pmatrix} 0 \\ 0 \\ -1.0 \end{pmatrix} v \tag{A.14}
$$

極配置によって，この線形システムの状態フィードバックを求めることができる．